

Tipi Algebrici (Record e Variant)

Definire nuovi tipi

In OCaml è possibile definire nuovi tipi di dato usando il costrutto `type`

```
In [1]: type data = int*int*int ;;
```

```
Out[1]: type data = int * int * int
```

Abbiamo definito un alias per `int*int*int` ...

```
In [2]: let controlla_data (d:data) =
  match d with
  | gg,mm,aaaa -> gg>0 && gg<=31
                && mm>0 && mm<=12
                && aaaa>1900 && aaaa<=2030 ;;
```

```
Out[2]: val controlla_data : data -> bool = <fun>
```

```
In [3]: let accoda_data (d:data) lis =
  if controlla_data(d) then d::lis else lis ;;
```

```
Out[3]: val accoda_data : data -> data list -> data list = <fun>
```

L'uso degli alias può rendere il codice più leggibile, soprattutto quando abbiamo a che fare con tipi molto complicati (tuple di tuple...). Se etichettiamo il parametro di una funzione con un alias, come in questi esempi, l'algoritmo di inferenza di tipi di OCaml lo utilizzerà rendendoci la descrizione del tipo inferito più comprensibile.

Tipi algebrici

OCaml consente di *comporre* tipi di dato per creare nuovi tipi

Immaginiamo un tipo come un insieme di valori

- `int` come l'insieme dei numeri interi,
- `bool` come l'insieme `{ true , false }`,
- ecc...

Le operazioni di costruzione di nuovi tipi in OCaml sono ispirate alle operazioni insiemistiche:

- *prodotto cartesiano* ($\times$), a cui corrispondono le *tuple* e i *record*
- *unione* ($\cup$) (che è una *somma* di insiemi), a cui corrispondo i *variant*

Tipi prodotto: tuple e record

Il prodotto cartesiano di due insiemi (es. $\mathbb{N} \times \mathbb{N}$) è un insieme di coppie.

- $(7, 4) \in \mathbb{N} \times \mathbb{N}$

Allo stesso modo il prodotto di due tipi (es. `int * int`) è un tipo di coppie

- `(7,4) : int*int`

Il discorso si estende a tuple di qualunque dimensione, come abbiamo già visto...

Record

I record sono un modo più avanzato di definire tipi prodotto

- sono simili a tuple, ma possiamo dare un nome agli elementi (detti *campi*)

Definiamo, ad esempio, un tipo record da usare per descrivere i punti nel piano cartesiano (cioè, in uno spazio bidimensionale). Ogni punto sarà descritto da una coordinata `x` e una coordinata `y` ambedue di tipo `float`.

```
In [4]: type punto_2d = { x: float; y: float; } ;;
```

```
Out[4]: type punto_2d = { x : float; y : float; }
```

Definito il tipo di un record, possiamo creare valori di quel tipo

```
In [5]: let p = { x = 3.; y = -4. } ;;
```

```
Out[5]: val p : punto_2d = {x = 3.; y = -4.}
```

Un valore di tipo record può essere decomposto usando il *pattern matching*

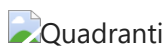
Ad esempio, calcoliamo il quadrante del piano cartesiano in cui ricade un punto

```
In [6]: let quadrante {x = x_pos; y = y_pos } =
      match x_pos>=0.,y_pos>=0. with
      | true,true -> 1
      | false,true -> 2
      | false,false -> 3
      | true,false -> 4 ;;

      quadrante {x=(-3.); y=2.};;
```

```
Out[6]: val quadrante : punto_2d -> int = <fun>
```

```
Out[6]: - : int = 2
```



ATTENZIONE: Nell'esempio ci sono due pattern matching. Quello che decompone il record è implicito nel `let` e usa il pattern `{x = x_pos; y = y_pos }`

Infatti, analogamente a quanto accade per le tuple, anche con i record è possibile specificare un pattern direttamente nell'operazione `let`, in modo tale da destrutturare immediatamente il record nelle sue componenti.

Un po' più semplicemente, possiamo usare direttamente `x` e `y` come nomi di variabili

```
In [7]: let quadrante {x; y} =
      match x>=0.,y>=0. with
```

```
| true,true -> 1
| false,true -> 2
| false,false -> 3
| true,false -> 4 ;;
```

Out[7]: val quadrante : punto_2d -> int = <fun>

Oppure possiamo accedere ai campi usando la *dot notation*: `p.x` e `p.y`

```
In [8]: let quadrante p =
  match p.x>=0.,p.y>=0. with
  | true,true -> 1
  | false,true -> 2
  | false,false -> 3
  | true,false -> 4 ;;
```

Out[8]: val quadrante : punto_2d -> int = <fun>

La dot notation consente di usare più variabili di tipo record in modo semplice

```
In [9]: let p = {x=3.; y=3.} ;;
let q = {x=6.; y=7.} ;;
p.x ;;
```

Out[9]: val p : punto_2d = {x = 3.; y = 3.}

Out[9]: val q : punto_2d = {x = 6.; y = 7.}

Out[9]: - : float = 3.

Esempio: Distanza tra due punti nel piano ($d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$)

```
In [10]: let distanza p1 p2 =
  sqrt ( (p2.x -. p1.x)**2. +. (p2.y -. p1.y)**2. ) ;;
```

Out[10]: val distanza : punto_2d -> punto_2d -> float = <fun>

```
In [11]: distanza p q ;;
```

Out[11]: - : float = 5.

Nella funzione `distanza` si è usata la funzione built-in `sqrt` che calcola la radice quadrata (già vista) e anche l'operatore `**` che calcola l'elevamento a potenza: dati `a` e `b` di tipo `float`, l'operazione `a**b` calcola `a` elevato alla `b`.

Quando si usano record con molti campi, le operazioni possono diventare verbose

```
In [12]: type persona = {nome: string; cognome: string; eta: int; indirizzo: string; citta: s
```

```
Out[12]: type persona = {
  nome : string;
  cognome : string;
  eta : int;
  indirizzo : string;
  citta : string;
}
```

```
In [13]: let mario = {nome="Mario"; cognome="Rossi"; eta=45; indirizzo="Via Roma 10"; citta="
```

```
Out[13]: val mario : persona =
  {nome = "Mario"; cognome = "Rossi"; eta = 45; indirizzo = "Via Roma 10";
   citta = "Pisa"}
```

Una piccola semplificazione è che possiamo definire una variabile in funzione di un'altra usando il costrutto di *functional update* with

```
In [14]: let bianca = { mario with nome = "Bianca"; eta=13 } ;;
```

```
Out[14]: val bianca : persona =
  {nome = "Bianca"; cognome = "Rossi"; eta = 13; indirizzo = "Via Roma 10";
   citta = "Pisa"}
```

ATTENZIONE: Stiamo sempre lavorando con strutture dati *immutabili*. Quindi, anche utilizzando la functional update quello che si otterrà è un nuovo record distinto dal precedente (che rimane come era prima).

Nota: record VS oggetti

I record assomigliano ai dizionari di JavaScript, e si potrebbe pensare che "assegnando" funzioni ai loro elementi quello che si ottiene siano oggetti (come accadrebbe in JavaScript).

```
In [15]: type finto_oggetto = { n: int; update: int -> int } ;;
```

```
Out[15]: type finto_oggetto = { n : int; update : int -> int; }
```

La differenza con gli oggetti è che *i record sono immutabili*

- le variabili non rappresentano uno "stato interno"... (base dell'object-orientation)

Inoltre i campi funzione (metodi) *non possono accedere* agli altri campi (variabili d'istanza)

Ad esempio:

```
In [16]: let obj = { n = 10; update = fun x -> x } ;;
obj.n = 11 ;; (* non posso assegnare... *)
```

```
Out[16]: val obj : finto_oggetto = {n = 10; update = <fun>}
```

```
Out[16]: - : bool = false
```

In questo esempio si è cercato di usare `=` per assegnare un nuovo valore al campo `n` del (finto) oggetto, come si farebbe in altri linguaggi. Qui chiaramente l'interprete ha risposto `false` in quanto l'operazione `=` è un confronto e con `obj.n = 11` non si è fatto altro che confrontare il valore di `n` con `11`.

Vedremo più avanti che nella parte imperativa di OCaml esistono alcuni operatori di assegnamento, tra cui `<-`. Se proviamo ad utilizzarlo in questo caso otteniamo come risposta perentoria che il campo del record è immutabile.

```
In [17]: obj.n <- 11 ;;
```

File "[17]", line 1, characters 0-11:

```
1 | obj.n <- 11 ;;
   ^^^^^^^^^^^
```

Error: The record field n is not mutable

Nell'esempio successivo, invece, vediamo che i campi funzione non possono accedere agli altri campi del record, come invece potrebbe fare un metodo di un (vero) oggetto.

In [18]:

```
let obj = { n = 10; update = fun x -> n + x } ;;
```

File "[18]", line 1, characters 38-39:

```
1 | let obj = { n = 10; update = fun x -> n + x } ;;
                                     ^
```

Error: Unbound value n

Tipi unione (o somma): variant

L'unione di due insiemi è... l'unione di due insiemi!

- $C = A \cup B$
- $x \in C$ se e solo se $x \in A$ oppure $x \in B$

L'unione di due tipi t_1 e t_2 è un nuovo tipo t che idealmente "include" tutti i valori di tipo t_1 e t_2

Mentre in alcuni linguaggi (ad es. TypeScript) si possono unire direttamente due tipi, scrivendo ad esempio:

TYPESCRIPT: `type t = string | int`

l'operazione di unione prevista da OCaml prevede di *etichettare* i valori dei tipi da unire

Ad esempio, per unire i tipi `string` e `int`, dobbiamo scegliere due etichette (ad esempio `Txt` e `Num`) e usarle in questo modo:

In [19]:

```
type numero_testo =
  | Txt of string
  | Num of int ;;
```

Out[19]: `type numero_testo = Txt of string | Num of int`

In sostanza, ogni riga della definizione corrisponde a un caso possibile di tipo di valore, ed è caratterizzato da una etichetta distinta (detta *costruttore*)

ATTENZIONE: I costruttori devono necessariamente iniziare con la lettera maiuscola

Anche i valori del nuovo tipo dovranno essere etichettati nello stesso modo:

In [20]:

```
let x = Txt "34" ;;
let y = Num 26 ;;
```

Out[20]: `val x : numero_testo = Txt "34"`

Out[20]: `val y : numero_testo = Num 26`

Dato un valore, è possibile ricostruire a quale dei tipi appartenga, tra quelli che sono stati uniti, usando i costruttori nel pattern matching

```
In [21]: match x with
| Txt s -> "testo"
| Num n -> "numero" ;;
```

```
Out[21]: - : string = "testo"
```

L'utilizzo dei costruttori `Txt` e `Num` nei pattern fa innanzitutto capire all'interprete di OCaml che la variabile `x` ha tipo `numero_testo`. A questo punto i vari pattern consentiranno di fare cose diverse in funzione del costruttore trovato in `x`.

In questo esempio, visto che `x` era associato al valore `Txt "34"`, il risultato ottenuto è la stringa `"testo"`.

Qualche esempio di funzione che usa il tipo `numero_testo`:

```
In [22]: let int_of_nt x =
  match x with
  | Txt s -> int_of_string s
  | Num n -> n ;;

let string_of_nt x =
  match x with
  | Txt s -> s
  | Num n -> string_of_int n ;;

int_of_nt (Txt "4") ;;
```

```
Out[22]: val int_of_nt : numero_testo -> int = <fun>
```

```
Out[22]: val string_of_nt : numero_testo -> string = <fun>
```

```
Out[22]: - : int = 4
```

Queste funzioni consentono semplicemente di estrarre una rappresentazione intera o testuale da un qualunque valore di tipo `numero_testo`, indipendentemente dal fatto che al suo interno sia usato il costruttore `Txt` seguito da una string, o il costruttore `Num` seguito da un numero.

Nella dichiarazione di un tipo unione, la parte `of <tipo>` è facoltativa

- si possono definire casi che consistono solo del costruttore
- sono come "singoletti" nell'insiemistica

Usando i soli costruttori, si possono definire *tipi enumerazione*

- In molti linguaggi chiamati *enum*

```
In [23]: type giorno = Lun | Mar | Mer | Gio | Ven | Sab | Dom ;;
```

```
Out[23]: type giorno = Lun | Mar | Mer | Gio | Ven | Sab | Dom
```

```
In [24]: Gio ;;
```

Out[24]: - : giorno = Gio

```
In [25]: let is_weekend g =
  match g with
  | Sab | Dom -> true
  | _ -> false ;;
```

Out[25]: val is_weekend : giorno -> bool = <fun>

Un esempio più complesso:

- tipo `int` esteso con `NaN`, $+\infty$ e $-\infty$

```
In [26]: type int_ext =
  | Num of int
  | NaN
  | Plus_inf
  | Minus_inf ;;
```

Out[26]: type int_ext = Num of int | NaN | Plus_inf | Minus_inf

In questo caso abbiamo combinato costruttori associati a un tipo (`Num of int`) con costruttori "singoletti" che descrivono casi particolari:

- `NaN` (sta per *Not a Number*) è il "numero" che si ottiene da un'operazione tra interi che restituisce un risultato non definito... ad esempio, la divisione $\frac{0}{0}$.
- `Plus_inf` rappresenta $+\infty$
- `Minus_inf` rappresenta $-\infty$

Definiamo un po' di operazioni aritmetiche sul nuovo tipo

```
In [27]: let sum x y =
  match x,y with
  | NaN,_ | _,NaN -> NaN
  | Plus_inf,Minus_inf | Minus_inf,Plus_inf -> NaN
  | Plus_inf,_ | _,Plus_inf -> Plus_inf
  | Minus_inf,_ | _,Minus_inf -> Minus_inf
  | Num n1,Num n2 -> Num (n1+n2) ;;
```

Out[27]: val sum : int_ext -> int_ext -> int_ext = <fun>

La somma di due `int_ext` deve tenere conto che uno o entrambi gli operandi potrebbero essere `NaN`, `Plus_inf` o `Minus_inf`. Nella funzione `sum`, i casi del pattern matching sono ordinati in modo da gestire prima tutti questi casi particolari, e lasciare come ultima opzione il caso "standard" in cui i due operandi sono in realtà degli interi normali.

ATTENZIONE: nessun warning: il pattern matching è esaustivo

- non abbiamo scordato casi!

Il controllo di esaustività dei pattern operato dal compilatore è utile per prevenire molti bug. È infatti impossibile scordarsi di considerare qualche caso (a meno che non ignoriamo il warning che otterremmo). Inoltre, il controllo fa sì che ci venga anche segnalato se abbiamo inserito dei casi ridondanti, ossia due pattern che possono fare match con lo stesso valore. Anche questo è spesso segnale di un errore di programmazione.

Ancora:

```
In [28]: let div x y =
  match x,y with
  | NaN,_ | _,NaN -> NaN
  | Plus_inf,Plus_inf | Minus_inf,Minus_inf -> NaN
  | Plus_inf,Minus_inf | Minus_inf,Plus_inf -> NaN
  | Plus_inf,_ -> Plus_inf
  | Minus_inf,_ -> Minus_inf
  | _,Plus_inf | _,Minus_inf -> Num 0
  | Num 0,Num 0 -> NaN
  | Num n,Num 0 -> if n>0 then Plus_inf else Minus_inf
  | Num n1,Num n2 -> Num (n1/n2) ;;
```

```
Out[28]: val div : int_ext -> int_ext -> int_ext = <fun>
```

L'implementazione dell'operazione di divisione è più complicata e critica dell'implementazione della somma. I casi da considerare sono di più, e soprattutto la divisione è un'operazione che può generare NaN, Plus_inf e Minus_inf a partire da operandi interi "normali"

```
In [29]: div (Num 3) (Num 0) ;;
```

```
Out[29]: - : int_ext = Plus_inf
```

3/0

```
In [30]: sum (Num 5) (div (Num 3) (Plus_inf)) ;;
```

```
Out[30]: - : int_ext = Num 5
```

5 + (3 / +∞)

Tipi opzione e tipi polimorfi

Un esempio di tipo variant molto utilizzato (e *built-in* in OCaml) è il tipo opzione

- consente di specificare un valore che può essere assente
- usa i costruttori `Some` e `None`

```
In [31]: Some 4;;
```

```
Out[31]: - : int option = Some 4
```

```
In [32]: None ;;
```

```
Out[32]: - : 'a option = None
```

È importante osservare il tipo di un valore di tipo opzione. Nel primo caso abbiamo `int option` (che in qualche modo significa "intero opzionale"), mentre nel secondo caso abbiamo `'a option`, in quanto il costruttore `None` è lo stesso qualunque sia il tipo che stiamo estendendo.

È lo stesso principio di funzionamento già visto per le liste... la lista `[4]` ha tipo `int list`, mentre la lista vuota `[]` ha tipo `'a list`.

I tipi opzione consentono di definire funzioni che restituiscono `None` in casi particolari

```
In [33]: let rec massimo lis =
  match lis with
  | [] -> None
  | x::lis' -> match massimo lis' with
    | None -> Some x
    | Some max -> if x>max then Some x
                  else Some max ;;
```

```
Out[33]: val massimo : 'a list -> 'a option = <fun>
```

La funzione `massimo` è polimorfa, in quanto l'unica operazione che viene eseguita sugli elementi della lista che prende come parametro è l'operazione di confronto `>` (che si applica a valori di qualunque tipo).

Questa funzione restituisce `None` in caso di lista vuota, in quanto il massimo di una lista priva di elementi non è definito.

```
In [34]: massimo [3;4;2] ;;
massimo ["albero";"cane";"bambino"] ;;
massimo [] ;;
```

```
Out[34]: - : int option = Some 4
```

```
Out[34]: - : string option = Some "cane"
```

```
Out[34]: - : 'a option = None
```

I tipi opzione sono (pre-)definiti in questo modo:

```
In [35]: type 'a option =
  | Some of 'a
  | None ;;
```

```
Out[35]: type 'a option = Some of 'a | None
```

questo esempio mostra come sia possibile definire *tipi polimorfi*

- usando le variabili di tipo `'a`, `'b`, ...

Le variabili di tipo sono utilizzate come veri e propri parametri nella definizione del tipo (*polimorfismo parametrico*).

Tipi record e variant insieme

Ovviamente, si possono creare tipi strutturati combinando a piacere record, variant, tuple, ecc... mostriamo giusto un esempio, per completezza, di punto multidimensionale che può rappresentare un punto nel piano (2 dimensioni) o nello spazio (3 dimensioni) usando diversi costruttori. Le coordinate del punto, in ognuno dei due casi, sono raccolte in un record:

```
In [36]: type punto_multidimensionale =
         | DueDim of {x: float; y: float; }
         | TreDim of {x: float; y: float; z: float; }
```

```
Out[36]: type punto_multidimensionale =
         DueDim of { x : float; y : float; }
         | TreDim of { x : float; y : float; z : float; }
```

```
In [37]: let p1 = DueDim {x=10.; y=10.} ;;
         let p2 = TreDim {x=4.; y=6.; z=8.} ;;
```

```
Out[37]: val p1 : punto_multidimensionale = DueDim {x = 10.; y = 10.}
```

```
Out[37]: val p2 : punto_multidimensionale = TreDim {x = 4.; y = 6.; z = 8.}
```

Tipi ricorsivi

Un'importante generalizzazione dei tipi unione sono i *tipi ricorsivi*

- sono tipi variant definiti in termini di se stessi

Vediamoli usati per definire il tipo di una lista di interi:

```
In [38]: type lista_di_int =
         | Nil
         | Elem of int * lista_di_int ;;

         let lst = Elem (3 ,Elem (4, Elem (6,Nil))) ;;
```

```
Out[38]: type lista_di_int = Nil | Elem of int * lista_di_int
```

```
Out[38]: val lst : lista_di_int = Elem (3, Elem (4, Elem (6, Nil)))
```

Una lista di interi è fatta di elementi di tipo `int` concatenati l'uno all'altro. Ogni elemento, quindi, deve prevedere anche un "riferimento" all'elemento successivo. L'ultimo elemento della lista avrà riferimento nullo (o meglio, a un elemento nullo), in quanto non esiste un successivo.

Il tipo `lista_di_int` deve quindi essere il tipo di una qualunque lista di interi (indipendentemente dalla sua lunghezza). Per questo è definito ricorsivamente...

Abbiamo due casi:

- la lista è vuota: possiamo rappresentarla con un singolo elemento nullo che rappresenteremo con il costruttore `Nil`
- la lista non è vuota: allora esiste un primo elemento che rappresenteremo con il costruttore `Elem`. Tale elemento prevederà un intero e un riferimento all'elemento successivo. L'elemento successivo, però, altri non è che il primo elemento di un'altra lista: la lista di tutti gli elementi che seguono l'elemento corrente. Quindi possiamo vedere il riferimento al prossimo elemento come un riferimento ad un'altra lista dello stesso tipo di quella che stiamo descrivendo. Per questo motivo al costruttore `Elem` è associato il tipo `int * lista_di_int`.

In un valore di tipo `lista_di_interi` in realtà non abbiamo dei veri e propri riferimenti tra gli elementi. Dal momento che la ricorsione è usata all'interno di una coppia `int * lista_di_int`, la concatenazione degli elementi sarà realizzata tramite *annidamento* di coppie, come negli esempi mostrati sopra.

Ora possiamo definire una funzione che corrisponde all'operatore `::` delle liste di OCaml

In [39]:

```
let cons x lis =
  Elem (x,lis) ;;

cons 5 lst;;
```

Out[39]: `val cons : int -> lista_di_int -> lista_di_int = <fun>`Out[39]: `- : lista_di_int = Elem (5, Elem (3, Elem (4, Elem (6, Nil))))`

E possiamo definire altre operazioni su `lista_di_int`:

In [40]:

```
let rec somma_lista lis =
  match lis with
  | Nil -> 0
  | Elem (x,lis') -> x + somma_lista lis' ;;

somma_lista ( Elem (3 ,Elem (4, Elem (6,Nil))));;
```

Out[40]: `val somma_lista : lista_di_int -> int = <fun>`Out[40]: `- : int = 13`

Il tipo built-in `'a list`, in effetti, è un caso particolare di variant ricorsivo polimorfo

In [41]:

```
type 'a my_list =
  | Empty (* corrisponde a [] *)
  | Cons of 'a * 'a my_list ;; (* corrisponde a _::_ *)
```

Out[41]: `type 'a my_list = Empty | Cons of 'a * 'a my_list`

Il tipo `'a my_list` qui presentato è una reimplementazione del tipo `'a list` di OCaml. La mostriamo solo per far vedere che `'a list` è effettivamente a tutti gli effetti un tipo variant ricorsivo polimorfo.

In [42]:

```
Empty ;;
Cons (3, Cons (4, Cons (1,Empty))) ;; (* corrisponde a 3::4::1 *)
Cons ("a", Cons ("b",Empty)) ;; (* corrisponde a "a"::"b"::[] *)
```

Out[42]: `- : 'a my_list = Empty`Out[42]: `- : int my_list = Cons (3, Cons (4, Cons (1, Empty)))`Out[42]: `- : string my_list = Cons ("a", Cons ("b", Empty))`

Per semplificare la lettura, OCaml usa una sintassi ad hoc per i valori di tipo `'a list`, che conosciamo bene: `[x1;x2;...]`

Tipi ricorsivi e... Alberi

Grazie ai tipi ricorsivi è semplice definire alberi

L'idea è esattamente quella già illustrata nell'esempio precedente delle liste di interi. Descriviamo ad esempio il tipo degli alberi binari con valori interi, `albero_bin`. Andiamo a descrivere le varie tipologie di elementi (in questo caso nodi intermedi o foglie) tramite costruttori diversi, e nel caso dei nodi intermedi (che sono collegati a sottoalberi) andiamo a riferirci ricorsivamente al tipo `albero_bin`.

```
In [43]: type albero_bin =
| Nodo of int*albero_bin*albero_bin
| Foglia of int ;;
```

```
Out[43]: type albero_bin = Nodo of int * albero_bin * albero_bin | Foglia of int
```

Definiamo un valore di tipo `albero_bin` un passo alla volta... dalle foglie alla radice:

```
In [44]: let n1 = Foglia 4 ;;
let n2 = Foglia 6 ;;
let n3 = Nodo (2,n1,n2) ;;
let n4 = Foglia 8 ;;
let n5 = Nodo (5,n3,n4) ;;
```

```
Out[44]: val n1 : albero_bin = Foglia 4
```

```
Out[44]: val n2 : albero_bin = Foglia 6
```

```
Out[44]: val n3 : albero_bin = Nodo (2, Foglia 4, Foglia 6)
```

```
Out[44]: val n4 : albero_bin = Foglia 8
```

```
Out[44]: val n5 : albero_bin = Nodo (5, Nodo (2, Foglia 4, Foglia 6), Foglia 8)
```

Vediamo alcune operazioni su alberi:

```
In [45]: let rec visita_ant a =
match a with
| Foglia v -> [v]
| Nodo (v,sx,dx) -> v::((visita_ant sx)@(visita_ant dx)) ;;

visita_ant n5 ;;
```

```
Out[45]: val visita_ant : albero_bin -> int list = <fun>
```

```
Out[45]: - : int list = [5; 2; 4; 6; 8]
```

```
In [46]: let rec somma_albero a =
match a with
| Foglia v -> v
| Nodo (v,sx,dx) -> v + somma_albero sx + somma_albero dx ;;

somma_albero n5 ;;
```

```
Out[46]: val somma_albero : albero_bin -> int = <fun>
```

```
Out[46]: - : int = 25
```

Tipi ricorsivi e... Abstract Syntax Tree (AST)

Tipo di albero utile per questo corso:

- gli alberi di sintassi astratta (*Abstract Syntax Tree - AST*)

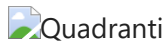
Risultato del *parsing* di un linguaggio formale:

- rappresentano la struttura sintattica di un programma/espressione/termine appartenente a un certo linguaggio
- linguaggio definito tramite una grammatica (es. formato BNF)

In JavaScript:

```
C ::= ... | if (E) C [else C] | ...
```

```
if (x>y) max=x else max=y
```



Esempio: le espressioni aritmetiche (su interi)

```
Exp ::= n | Exp op Exp | -Exp | (Exp)
op ::= + | - | * | / | %
```

La rappresentazione come albero di sintassi astratta si basa sulla definizione di un tipo variant con un costruttore per ogni produzione (caso) della grammatica in formato BNF. Possiamo non rappresentare le parentesi: le parentesi servono infatti solo per capire quale operatore si applichi per primo, e questo viene rappresentato dalla posizione dei corrispondenti nodi nell'AST.

```
In [47]: type op = Add | Sub | Mul | Div | Mod ;;
         type exp =
           | Val of int
           | Op of op*exp*exp
           | UMin of exp ;;
```

```
Out[47]: type op = Add | Sub | Mul | Div | Mod
```

```
Out[47]: type exp = Val of int | Op of op * exp * exp | UMin of exp
```

Ogni costruttore è associato a un tipo che rappresenta le informazioni espresse dalla corrispondente produzione nella grammatica. Ad esempio, il costruttore `Val`, che rappresenta la produzione `Exp ::= n` è associato al tipo `int` (per poter memorizzare `n`), mentre il costruttore `Op`, che rappresenta la produzione `Exp ::= Exp op Exp`, è associato al tipo `op*exp*exp` in modo da poter memorizzare il simbolo dell'operatore e le due espressioni a cui tale operatore si applica (che sono quindi sottoalberi dell'AST).

Sintassi astratta:

```
3 * 7 - 5
```

```
In [48]: let exp1 = Op (Sub, (Op (Mul, Val 3, Val 7)), Val 5) ;;
```

```
Out[48]: val exp1 : exp = Op (Sub, Op (Mul, Val 3, Val 7), Val 5)
```

```
-3 * (7-5)
```

```
In [49]: let exp2 = Op (Mul, UMin (Val 3), (Op (Sub, Val 7, Val 5))) ;;
```

```
Out[49]: val exp2 : exp = Op (Mul, UMin (Val 3), Op (Sub, Val 7, Val 5))
```

Per fare un paio di esempi di uso degli AST rappresentati come tipi algebrici in OCaml, vediamo una funzione che trasforma l'AST di una espressione in una stringa, e una funzione che calcola il risultato (valuta) una espressione.

Rappresentazione come stringa:

```
In [50]: let symbol o =
  match o with
  | Add -> "+" | Sub -> "-" | Mul -> "*" | Div -> "/" | Mod -> "%" ;;
let rec to_string e =
  match e with
  | Val n -> string_of_int n
  | Op (o,e1,e2) -> "(" ^ (to_string e1) ^ (symbol o) ^ (to_string e2) ^ ")"
  | UMin e' -> "(-" ^ (to_string e') ^ ")" ;;
```

```
Out[50]: val symbol : op -> string = <fun>
```

```
Out[50]: val to_string : exp -> string = <fun>
```

```
In [51]: to_string exp1 ;;
to_string exp2 ;;
```

```
Out[51]: - : string = "((3*7)-5)"
```

```
Out[51]: - : string = "((-3)*(7-5))"
```

Valutazione delle espressioni:

```
In [52]: let rec eval e =
  match e with
  | Val n -> n
  | Op (o,e1,e2) ->
    (match o with
    | Add -> (eval e1) + (eval e2)
    | Sub -> (eval e1) - (eval e2)
    | Mul -> (eval e1) * (eval e2)
    | Div -> (eval e1) / (eval e2)
    | Mod -> (eval e1) mod (eval e2)
    )
  | UMin e' -> - (eval e') ;;
```

```
Out[52]: val eval : exp -> int = <fun>
```

```
In [53]: eval exp1 ;;
eval exp2 ;;
```

```
Out[53]: - : int = 16
```

```
Out[53]: - : int = -6
```